

Selenium WebDriver Assignment Report

Course: CSE430 - Software Testing and Quality Assurance Lab

Name: Akhlak Hossain

Student ID: 2022-3-60-057

Instructor: Shartaz Sajid Nahid

August 5, 2026

Assignment Overview

This report details the implementation of a Python script using Selenium WebDriver to automate interactions and run tests on the target web application: <https://books.toscrape.com>. The script was executed successfully in a headless Google Colab environment.

Task 1: Setup and Basic Assertions

Approach: A headless Chrome WebDriver was initialized and navigated to the target URL. The script printed the page title and URL, then evaluated three basic assertions regarding the page state using a custom logging function.

```
driver.get("https://books.toscrape.com")
page_title = driver.title
log('Title contains "Books"', "Books" in page_title)
```

Results: All assertions passed. The script confirmed that the title contains “Books”, the URL starts with “https://”, and the title remains unchanged after triggering a page refresh.

Task 2: Element Locators

Approach: Four different WebDriver locator strategies were utilized to find specific elements on the homepage, demonstrating the ability to target the DOM via various attributes.

```
titles_by_class = driver.find_elements(By.CLASS_NAME, "
    product_pod")
star_ratings = driver.find_elements(By.CSS_SELECTOR, ".star-
    rating")
next_button = driver.find_elements(By.XPATH, "//li[@class='next
    ']/a")
articles = driver.find_elements(By.TAG_NAME, "article")
```

Results: All locator assertions passed. The log confirmed that element counts were greater than zero, specifically finding 20 book containers, 20 star-rating elements, 1 “next” button, and 20 <article> elements.

Task 3: Data Scraping

Approach: The script iterated through pages 1 through 4 of the catalogue. It systematically extracted the title, price, star rating, and availability text for each book, formatted the data, and exported it to a CSV file.

```
for page in range(1, 5):
    url = f"https://books.toscrape.com/catalogue/page-{page}.html"
    driver.get(url)
    # Element extraction loop follows...
```

Results: The script successfully collected 80 total books, satisfying the quantity assertion. The data was saved to `books_data.csv`, and a pandas dataframe preview accurately displayed the first 3 rows (“A Light in the Attic”, “Tipping the Velvet”, and “Soumission”).

Task 4: Detail Page Validation

Approach: The script selected the first 3 scraped books, navigated to their individual detail pages, and performed cross-reference assertions to ensure the detail page content matched the previously scraped catalogue data.

```
detail_title = driver.find_element(By.TAG_NAME, "h1").text
log(f"Detail title matches for '{book['title']}' ", book['title']
    in detail_title or detail_title in book['title'])
```

Results: The script evaluated 5 assertions per book. For all three test cases, it successfully verified that the title was not empty, the detail title matched the scraped title, the price matched, and the availability text contained “In stock”.

Note on Failures: The assertion verifying that the “Add to basket” button was present and enabled failed for all three books. Because the script was running in a headless Colab environment, UI elements like buttons often require explicit scrolling or altered CSS targeting to register as “interactable” by Selenium’s `is_enabled()` method.

Task 5: Final Summary

A custom helper function, `log(test_name, passed)`, was used throughout the execution to prevent the notebook from crashing upon failed assertions.

Final Result: The notebook executed from top to bottom, recording a final summary of 20 out of 23 assertions passed.